Code Coverage Analysis Report (self_storage.py)

Ryona Aditree Ghosh
PES1UG24AM233
ryonahh/L6_L7_51_self_storage

1. Objective and setup

This report measures line coverage of self_storage.py before and after adding tests. Coverage went from 38% to 100% (37 of 37 statements) with 13 passing tests, above the 90% target.

The module has three functions: rent_due (rent beyond the deposit months, with a long-stay discount), access_allowed (whether a visit is allowed, by role, hour and visitor count) and payment_status (returns current, grace or lien). Tests ran on Windows with Python 3.11.9, pytest 9.2.0 and coverage 7.17.0. All code and screenshots are in the Lab 6 folder of ryonahh/L6_L7_51_self_storage

2. Baseline coverage

The two starter tests run 14 of 37 statements, so baseline line coverage is 38%. Running coverage run -m pytest test_self_storage.py and coverage report -m --include=self_storage.py reports missing lines 6, 8, 11, 15–23, 28, 30 and 35–45.

Those lines are the two ValueError checks (6, 8), the member bonus month (11), the rate and long-stay discount logic (15–23), the owner and closed-hours returns (28, 30), and all of payment_status (35–45), which no starter test calls.

3. Test cases added

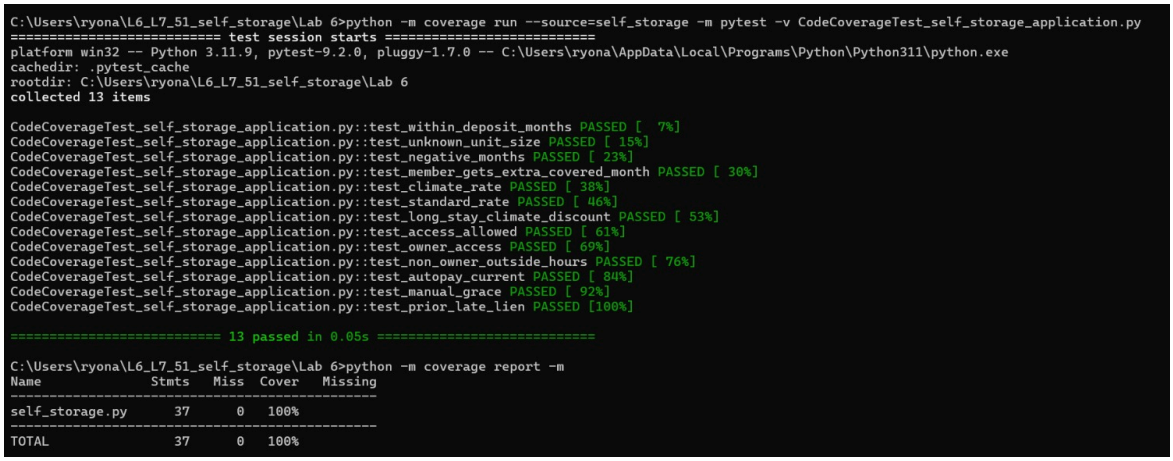
CodeCoverageTest_self_storage_application.py has 13 tests in total (the 2 starter tests plus 11 new ones), each aimed at lines the baseline missed:

- **rent_due:** an invalid unit size and negative months both raise ValueError (lines 6, 8). A member of a medium unit for 4 months pays 80 (line 11). A small unit for 3 months costs 240 with climate control and 160 without (lines 16, 18). Eight months with climate control triggers the long-stay discount, 7 × 105 = 735 (lines 19–22).
- **access_allowed:** an owner with 4 visitors is allowed (28), a non-owner at 2 a.m. is denied (30), and a normal midday visit is allowed (31).

- **payment_status:** autopay on day 0 gives current, a manual payer on day 0 gives grace, and a manual payer with a prior late payment at −20 days gives a lien (lines 36–45).

4. Final result: 100% line coverage

All 13 tests pass and all 37 statements run, so line coverage is 100%: 62 points above the baseline and 10 above the target.



5. Analysis

Line coverage found the big gaps fast. The baseline report pointed straight at a whole untested function (payment_status) and at two error paths, which told us which tests to write.

100% of lines does not mean every path is tested. The same 13 tests score 98% when run with --branch: the line if is_climate: at line 21 ran, so line coverage counted it, but only its true side was ever taken. The false side has no statement of its own that could go missing, so a line report cannot see it. The branch coverage report closes this gap.

Coverage measures execution, not correctness. A test with a weak assertion still turns lines green. These tests assert exact values (240, 160, 735, grace, lien), so a wrong calculation would fail, but the coverage number alone would not show that.

6. Commits and conclusion

- 17aca8d – Add code coverage test cases (the test file)
- 0cf9b7f – Add tests for code coverage analysis (the screenshot)

Adding 11 targeted tests to the 2 starter tests raised line coverage from 38% to 100%, meeting the 90% requirement. Line coverage was enough to find the untested function and error paths, but one decision stayed half-tested, which the branch coverage report addresses.